

Departamento de Ingeniería
Telemática y Electrónica

Escuela Técnica Superior de Ingeniería y
Sistemas de Telecomunicación

Universidad Politécnica de Madrid



Diseño Digital 1

AR3
Ejercicios de modelado y simulación de sistemas secuenciales simples

Introducción.

En este documento se proponen una serie de ejercicios de modelado y simulación VHDL de circuitos secuenciales simples.

1.- Flip-flops

Los *flip-flops* son las células digitales comúnmente utilizadas para materializar la memoria de los sistemas digitales síncronos cableados. Disponen de una entrada de reloj, de entradas de control síncrono (su número y modo de funcionamiento depende del tipo de *flip-flop*) y de una entrada de inicialización asíncrona (*preset* o *reset*, normalmente activa a nivel bajo). Su funcionamiento genérico se resume a continuación:

- Cuenta con dos modos de funcionamiento: síncrono (gobernado por la entrada de reloj) y asíncrono (gobernado por la entrada de inicialización asíncrona). El segundo modo (asíncrono) tiene prioridad sobre el primero, es decir, si la entrada de inicialización asíncrona está activa el *flip-flop* no atiende a la entrada de reloj.

- La entrada de inicialización asíncrona, normalmente activa a nivel bajo, es activa *por nivel*. Se emplea para inicializar el estado de memoria del flip-flop en el arranque del sistema. Si es una entrada de *reset* (también denominada *clear*), pone a 0 la salida del flip-flop; si es una entrada de *preset* lo pone a 1. Mientras se mantiene activa esta entrada, el modo de funcionamiento síncrono permanece inhibido.

- El modo de funcionamiento síncrono es el que rige la operación del flip-flop después del arranque del sistema. El flip-flop es sensible a uno de los flancos del reloj (normalmente al de subida), dicho flanco se denomina *el flanco activo* y es el que define los instantes en que la salida del *flip-flop* (el estado de memoria) puede cambiar –de los flip-flops se dice también que *se disparan* (su operación *se dispara*…) por flanco de subida o bajada; debe entenderse que, en el modo de funcionamiento síncrono, la salida puede cambiar únicamente en los instantes en que ocurre el flanco de reloj al que es sensible el *flip-flop*.

- El nivel lógico del estado de memoria que se establece en la salida del *flip-flop* tras la ocurrencia de un flanco activo de reloj depende del nivel lógico de las entradas síncronas del flip-flop y del nivel de su salida (del estado de memoria) antes del flanco. Por ejemplo:

    - Un *flip-flop* tipo D tiene una única entrada síncrona, D. En los flancos activos de reloj la salida del *flip-flop* toma el nivel lógico presente en la entrada D –en este tipo de *flip-flop*, el más comúnmente empleado (prácticamente el único, hoy en día), el cambio en el estado de memoria no depende directamente de la salida del *flip-flop* antes del flanco; es el único tipo de *flip-flop* con esta propiedad y, por ello, el de funcionamiento más simple.

    - Un *flip-flop* tipo T tiene una única entrada síncrona, T. En los flancos activos de reloj, si T vale 1 la salida del *flip-flop* conmuta; si T vale 0, la salida se mantiene, no cambia. En este *flip-flop*, a diferencia del anterior, la transición en el estado de memoria del *flip-flop* sí depende del estado de la salida antes del flanco activo

Antiguamente se empleaban distintos tipos de flip-flops (D, T, J-K y R-S), pero como con cualquiera de ellos se puede realizar cualquier sistema digital síncrono y como el desarrollo de la tecnología digital convirtió en despreciables las ventajas[1] que ofrecía el uso de los de operación más compleja (los T, R-S y J-K), hace ya bastantes años que sólo se emplean los *flip-flops* tipo D, los que tienen una interfaz y un funcionamiento más sencillo, para la realización de sistemas digitales cableados.

Modelado y simulación VHDL

El siguiente código VHDL modela un flip-flop tipo D:

```
library ieee;
use ieee.std_logic_1164.all;

entity ff_D is
port(clk:  in     std_logic;
     nRst: in     std_logic;
     D:    in     std_logic;
     Q:    buffer std_logic);

end entity;

architecture rtl of ff_D is
begin
  process(clk, nRst)
  begin
    if nRst = '0' then
      Q <= '0';

    elsif clk'event and clk = '1' then
      Q <= D;

    end if;
  end process;
end rtl;
```

Recuerde que:

- Con un proceso VHDL sólo pueden modelarse lo que denominamos circuitos secuenciales síncronos simples –los que tienen una arquitectura de Moore y en los que, además, la salida del circuito se obtiene directamente del estado de memoria (de la salida de los flip-flops); esta clase de circuitos incluye algunos de los bloques secuenciales más comúnmente utilizados: registros, contadores (salvo los que tienen salida de fin de cuenta) y *flip-flops*.

- El proceso que modela un secuencial simple sólo debe ser sensible a la entrada de reloj y a la entrada de inicialización asíncrona; El código del proceso se construye siempre con una sentencia *if* en la que:

[1] El ahorro de un puñado de células combinacionales en la lógica de cálculo de transiciones de estado de los circuitos secuenciales.

- primero (*if*) se evalúa la actividad de la entrada de inicialización asíncrona y, de confirmarse, se proyecta su efecto sobre la salida de los *flip-flops* del circuito modelado,
- y después, alternativamente (*elsif*), se evalúa la ocurrencia de un flanco activo de reloj y, supeditado a dicha ocurrencia, se especifica el valor que toma la salida del circuito en función de las entradas *síncronas* y el propio estado de la salida.

Debe evitar errores de estilo de los que quepa inferir ignorancia sobre el procedimiento de modelado o el funcionamiento del hardware modelado, como, por ejemplo:

- Incluir entradas síncronas en la lista de sensibilidad
- Explicitar combinaciones de las entradas síncronas que no producen cambios en la salida del circuito

Para la realización de *test-benches* de modelos de circuitos secuenciales, recuerde que:

- Tiene que definir una constante (T_clk) para establecer el valor del periodo de la señal de reloj que se utiliza como estímulo en la simulación
- Tiene que generar el reloj en un proceso de ejecución cíclica
- Para temporizar los estímulos síncronos y obtener un retardo de un ciclo de reloj debe emplear sentencias *wait until* clk*'event and* clk = *'1'*
- Cuando tenga que generar retardos de varios periodos de reloj, utilice una sentencia *wait for* seguida por una sentencia *wait until*
- Al inicio del proceso que define la secuencia de estímulos de simulación, comience realizando una inicialización asíncrona del circuito; cuando desactive la entrada de inicialización asíncrona todas las entradas síncronas tienen que tener ya definido un nivel lógico no indeterminado.

Ejercicios

Ejercicio 1.1

Realice el modelo de un *flip-flop* tipo D con entrada de reset[2] asíncrono y entrada de habilitación. Realice un *test-bench* para verificar el modelo y ejecute una simulación para comprobar su corrección.

Ejercicio 1.2

Nota: La realización de éste y los siguientes ejercicios puede acelerarse si reaprovecha el código de los ejercicios que vaya realizando.

Realice el modelo de un *flip-flop* tipo D con entrada de reset asíncrono, entrada de habilitación y entrada de reset síncrono supeditada a la habilitación. Realice un *test-bench* para verificar el modelo y ejecute una simulación para comprobar su corrección.

[2] Todas las entradas asíncronas serán activas a nivel bajo; las entradas síncronas serán, salvo que se indique los contrario, activas a nivel alto.

Ejercicio 1.3

Realice el modelo de un *flip-flop* tipo D con entrada de reset asíncrono, entrada de habilitación y entrada de reset y preset síncrono independientes de la habilitación; el reset es prioritario sobre el preset. Realice un *test-bench* para verificar el modelo y ejecute una simulación para comprobar su corrección.

Aunque usted no tendrá oportunidad, ni necesidad, de utilizar *flip-flops* J-K o R-S, no está de más que conozca su funcionamiento; los dos siguientes ejercicios le brindan una oportunidad para ello y, al mismo tiempo, le pueden servir para consolidar sus conocimientos y habilidades para el modelado y simulación de sistemas secuenciales síncronos simples.

Ejercicio Opcional 1.4

Un *flip-flop J-K* cuenta con dos entradas síncronas, *J* y *K* que determinan el siguiente modo de funcionamiento:

- Si *J* y *K* valen 0, el estado de memoria (la salida del *flip-flop*) se mantiene invariable tras el flanco activo de reloj
- Si *J* y *K* valen 1, el estado de memoria (la salida del *flip-flop*) se invierte tras el flanco activo de reloj
- Si *J* vale 1 y *K* vale 0, el estado de memoria (la salida del *flip-flop*) se pone a 1 tras el flanco activo de reloj
- Si *J* vale 0 y *K* vale 1, el estado de memoria (la salida del *flip-flop*) se pone a 0 tras el flanco activo de reloj

Realice el modelo de un *flip-flop J-K* con entrada de reset asíncrono. Realice un *test-bench* para verificar el modelo y ejecute una simulación para comprobar su corrección.

Ejercicio Opcional 1.5

Un *flip-flop R-S* cuenta con dos entradas síncronas, *R* (reset) y *S* (preset) que determinan el siguiente modo de funcionamiento:

- Si *R* y *S* valen 0, el estado de memoria (la salida del *flip-flop*) se mantiene invariable tras el flanco activo de reloj
- Si *R* vale 1 y *S* vale 0, el estado de memoria (la salida del *flip-flop*) se pone a 0 tras el flanco activo de reloj
- Si *R* vale 0 y *S* vale 1, el estado de memoria (la salida del *flip-flop*) se pone a 0 tras el flanco activo de reloj
- Está prohibido que ambas entradas puedan valer 1 al mismo tiempo.

Realice el modelo de un *flip-flop R-S* con entrada de preset asíncrono. Realice un *test-bench* para verificar el modelo y ejecute una simulación para comprobar su corrección.

2.- Registros

Los registros son circuitos secuenciales de uso muy común y cuyo diseño lógico puede abordarse aplicando técnicas de escalado, en las que una célula básica, construida en torno a un *flip-flop*, se conecta con otras idénticas siguiendo un determinado patrón estructural.

Tienen una estructura que encaja en el modelo de arquitecturas de Moore en las que las salidas son directamente el estado de memoria[3] y, por tanto, pueden modelarse en un proceso VHDL con una estructura semejante a la empleada en el modelado de *flip-flops*.

Registros

Los registros son subsistemas que se emplean fundamentalmente para el almacenamiento temporal de datos, aunque también tienen otras aplicaciones, como la paralelización de datos recibidos en serie, o la función inversa, la serialización de datos; también se usan para generar o detectar secuencias, como células de retardo, o para la segmentación de rutas de datos.

Disponen siempre de puertos para el ingreso y extracción de datos y suelen contar, además, con entradas síncronas de inicialización (reset, preset o load), entradas de habilitación o, cuando cuentan con una estructura versátil que les otorga de la posibilidad de disponer de diversos modos de funcionamiento, con entradas de control de modo.

Se clasifican atendiendo al modo en que se escriben y leen los datos en ellos, y su descripción completa debe especificar, además, el número de bits del registro y las entradas síncronas de que disponen.

Modelado VHDL

El modelado de los registros se realiza empleando los esquemas y elementos que se utilizan para el modelado de los *flip-flops*. El único elemento novedoso, que rara vez se usa en el modelado de circuitos combinacionales y nunca en el de *flip-flops*, es el operador de encadenamiento, *&*, que se usa muy frecuentemente para modelar registros de desplazamiento; por ejemplo:

```
reg(7 downto 0) <= reg(6 downto 0)&D_in;
```

Ejercicios

Ejercicio 2.1

Realice el modelo de un registro[4] de 8 bits con entrada de reset[5] asíncrono y entrada de habilitación. Realice un *test-bench* para verificar el modelo y ejecute una simulación para comprobar su corrección.

Ejercicio 2.2

[3] Con la excepción de los contadores con salida de fin de cuenta que, dependiendo del caso, tienen una arquitectura de Mealy o de Moore y, en cualquier caso, incorporan lógica combinacional interpuesta entre el estado de memoria y la señal de fin de cuenta.

[4] Si no se indica nada, debe suponerse que se trata de un registro con escritura y lectura de datos en paralelo.

[5] Todas las entradas asíncronas serán activas a nivel bajo; las entradas síncronas serán, salvo que se indique los contrario, activas a nivel alto.

Realice el modelo VHDL de un registro de desplazamiento de 8 bits con salida de datos en paralelo, entrada de datos serie y entrada de control de modo de operación que determina el sentido del desplazamiento (a izquierda o derecha), El registro debe contar además con una entrada de reset asíncrono y una entrada de habilitación. Realice un *test-bench* para verificar el modelo y ejecute una simulación para comprobar su corrección.